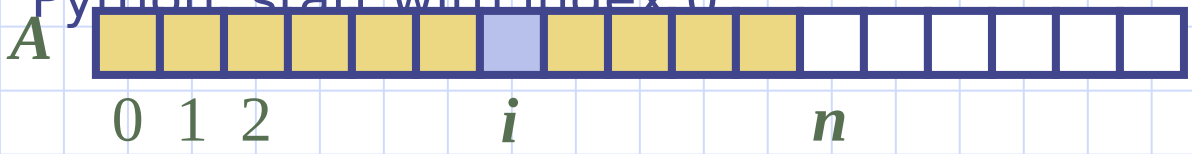
Insertion Sort and Introduction to Complexity

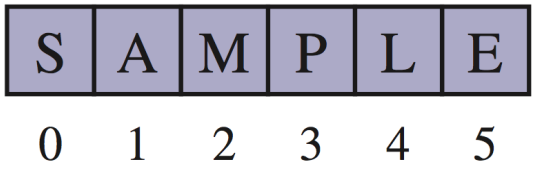
Python Sequence Classes

- Python has built-in types, **list**, **tuple**, and **str**.
- Each of these **sequence** types supports indexing to access an individual element of a sequence, using a syntax such as A[i]
- Each of these types uses an **array** to represent the sequence.
  - An array is a set of memory locations that can be addressed using consecutive indices, which, in Python, start with index 0

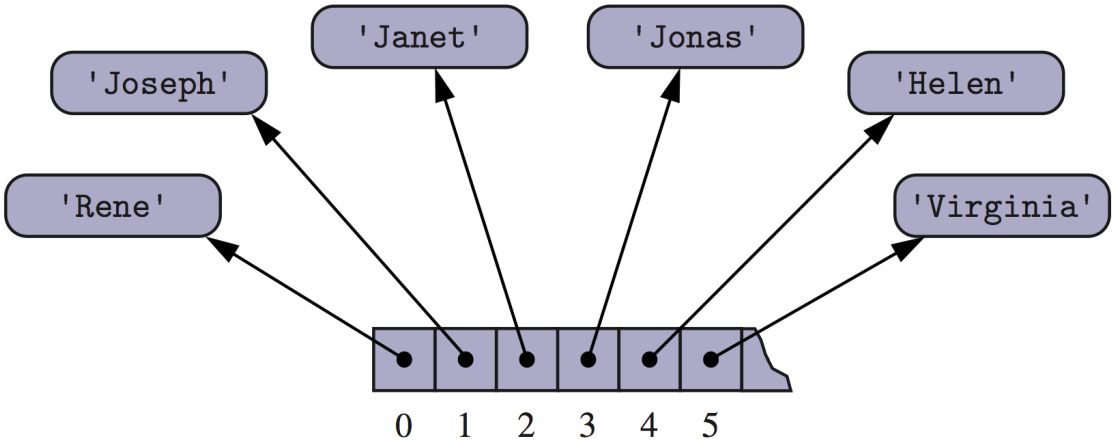


Arrays of Characters or Object References

- An array can store primitive elements, such as characters, giving us a **compact array.**



- An array can also store references to objects.



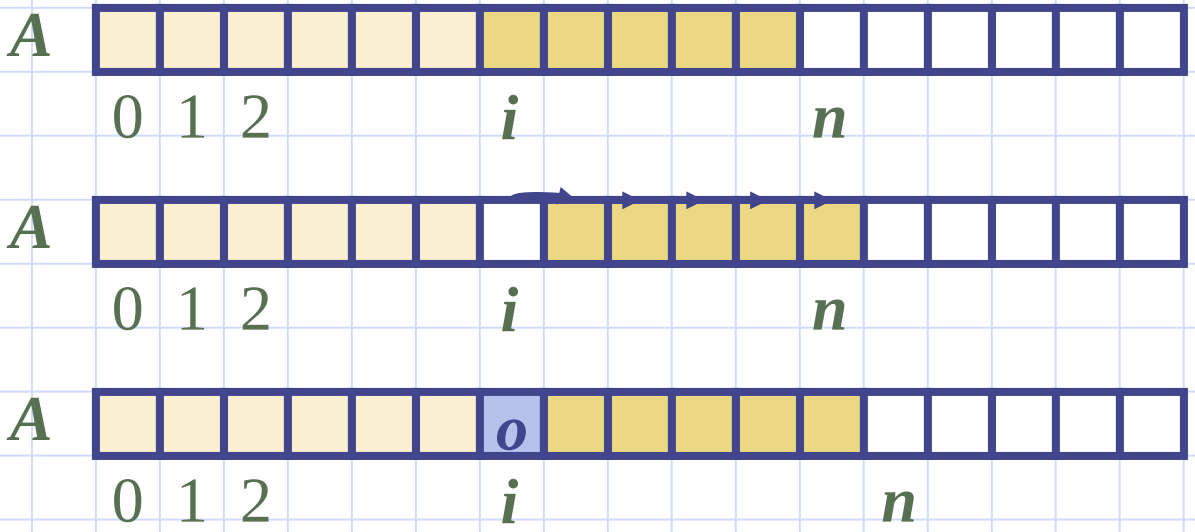
Compact Arrays

- Primary support for compact arrays is in a module named **array**.
  - That module defines a class, also named array, providing compact storage for arrays of primitive data types.
- The constructor for the array class requires a type code as a first parameter, which is a character that designates the type of data that will be

```
primes = array('i', [2, 3, 5, 7, 11, 13, 17, 19])
```

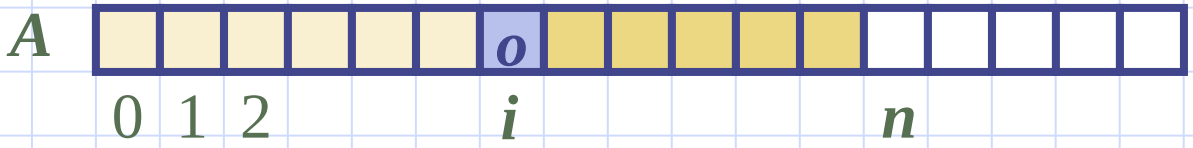
Insertion

- In an operation $add(i, o)$, we need to make room for the new element by shifting forward the $n - i$ elements $A[i], \ldots, A[n-1]$
- In the worst case ($i = 0$), this takes $O(n)$ time



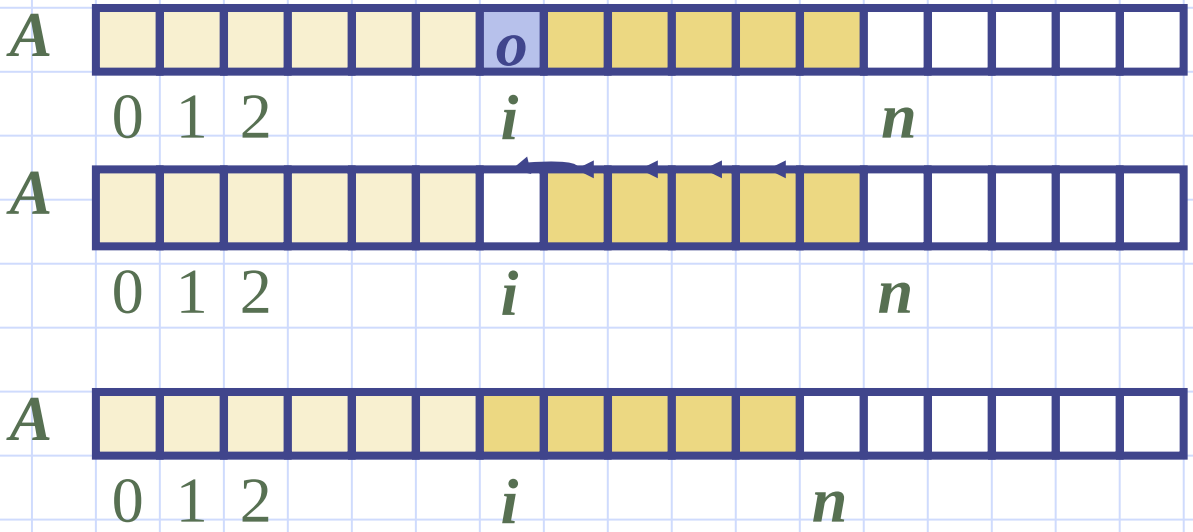
Insertion

- How many elements need to be moved? It depends on the index *i*
- This is called the *complexity* of an operation. We (almost) always talk about the *worst-case complexity*.
- In the worst case ($i = 0$), this takes $O(n)$ time, where *n* is the length of the array. This length changes depending on the input, so we use a variable *n*. We will understand soon what *O(n)* means.



Element Removal

- In an operation ***remove***(i), we need to fill the hole left by the removed element by shifting backward the $n - i - 1$ elements $A[i + 1], \ldots, A[n - 1]$
- In the worst case ($i = 0$), this takes $O(n)$ time



Insertion sort

Algorithm InsertionSort(A):

Input: An array A of n comparable elements

Output: The array A with elements rearranged in nondecreasing order

for k from 1 to n − 1 **do**

Insert A[k] at its proper location within A[0], A[1], . . ., A[k].

Code Fragment 5.9: High-level description of the insertion-sort algorithm.

- Elements must be comparable. For example, given two elements a and b, we should be able to say whether a>b, or a<b, or a=b.
- They can be integers, floating point numbers, or strings that can are lexicographically comparable.

Insertion sort

Algorithm InsertionSort(A):

Input: An array A of n comparable elements

Output: The array A with elements rearranged in nondecreasing order

for k from 1 to n − 1 **do**

Insert A[k] at its proper location within A[0], A[1], . . ., A[k].

Code Fragment 5.9: High-level description of the insertion-sort algorithm.

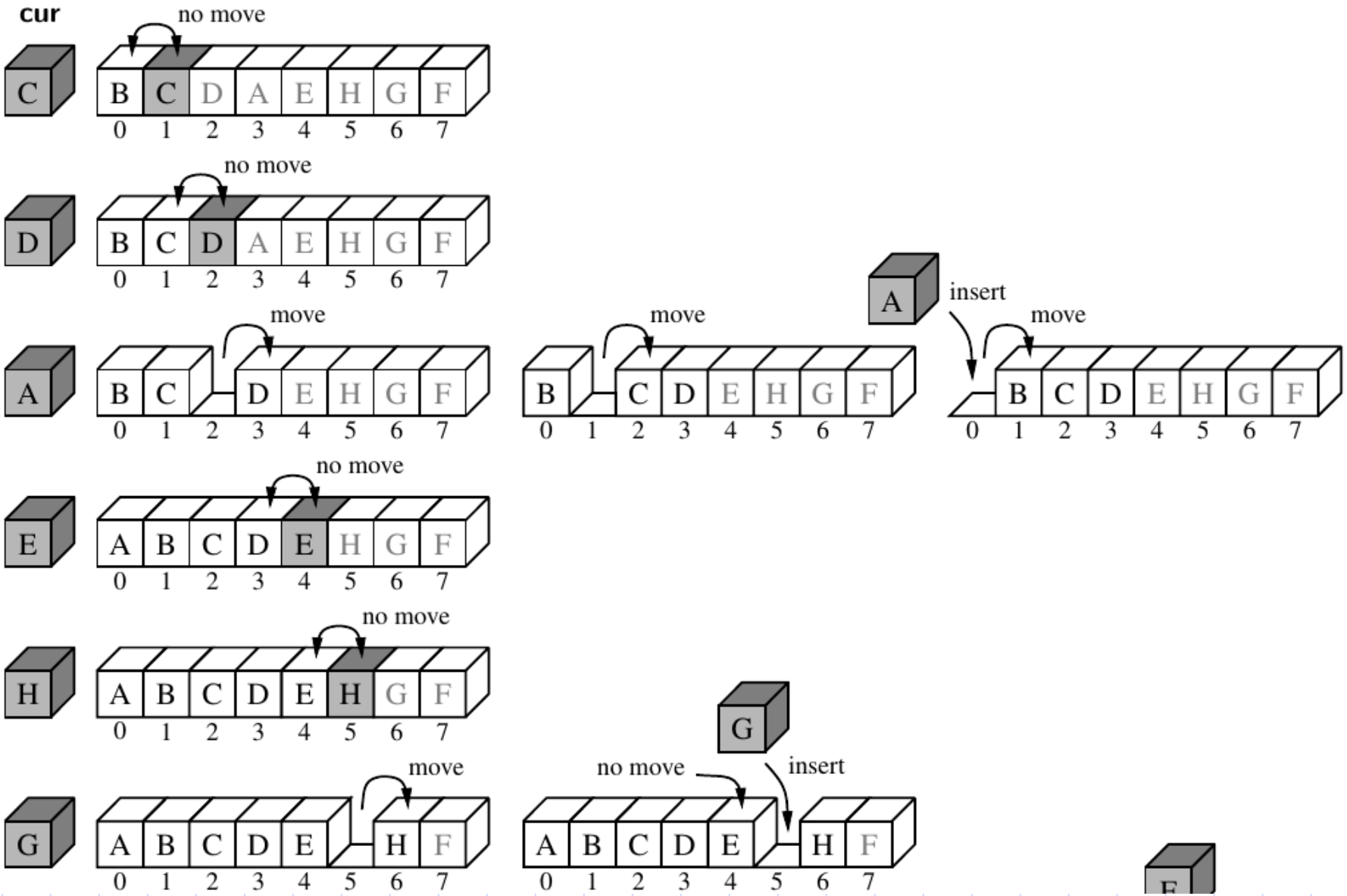
- We start from the second element of the array, and try to place it correctly so that the two elements (first(a) and second(b)) are sorted.
- If a=b, we do nothing. If a<b, we do nothing, because in both cases a and b are in non-decreasing order. If a>b, we swap a and b.
-

Insertion sort

```
1  def insertion_sort(A):
2    """Sort list of comparable elements into nondecreasing order."""
3    for k in range(1, len(A)):                  # from 1 to n-1
4      cur = A[k]                                # current element to be inserted
5      j = k                                     # find correct index j for current
6      while j > 0 and A[j−1] > cur:             # element A[j-1] must be after curren
7        A[j] = A[j−1]
8        j −= 1
9      A[j] = cur                                # cur is now in the right place
```

Code Fragment 5.10: Python code for performing insertion-sort on a list.

- The index of the current element to be inserted in its correct position is k
- We push it to the left as far as possible and then insert it in the correct position.


cur
no move
C
B C D A E H G F
0 1 2 3 4 5 6 7
no move
D
B C D A E H G F
0 1 2 3 4 5 6 7
move
A
B C D E H G F
0 1 2 3 4 5 6 7
move
B C D E H G F
0 1 2 3 4 5 6 7
A
insert
move
B C D E H G F
0 1 2 3 4 5 6 7
no move
E
A B C D E H G F
0 1 2 3 4 5 6 7
no move
H
A B C D E H G F
0 1 2 3 4 5 6 7
move
G
A B C D E H F
0 1 2 3 4 5 6 7
G
no move
insert
A B C D E H F
0 1 2 3 4 5 6 7

Complexity of Insertion sort

We are doing the insertion operation again and again. We have to consider the worst-case complexity. Why?

Because there will be no surprise if we can say what will happen in the worst case.

Though it is possible to analyse what might happen on an average, we will not consider *average case complexity* in this unit.

Insertion sort - complexity

We are performing the insertion operation again and again, how many times? We start from the second element, so we are doing this *n-1* times if the array is of length *n*

We will pretend the array is very large, because we don’t know the size of the input beforehand.

So, there is no difference between a very large *n* and *n-1.* 999999 is as large as 1000000

We consider *asymptotic complexity.*

Insertion sort - complexity

What is the complexity of a single insertion operation? It depends on two things.

How far is the element from the start of the array, and how far we have to move it before inserting.

We have noticed that sometime the element is already in its correct position, and we don’t need to do anything.

But that is not the worst case. We will consider the maximum amount of work we need to do for a single insertion operation.

Insertion sort - complexity

Since the length of the array is *n*, and we have to do *n* comparisons for the insertion operation, we can say each insertion takes *n* time.

So the total time is nXn =n^2

But wait! That is not true. We don't do *n* work for each insertion. We did just one comparison for the second element, two for the third element …. (n-1) comparisons for the last element.

So the total work is:
1+2+3+…+(n-1)=n(n-1)/2=n^2/2 – n/2

Insertion sort - complexity

So the total work is:
1+2+3+...+(n-1)=n(n-1)/2=n^2/2 – n/2

Remember, we are considering very large n. if n= 100, n^2/2= 5000, n/2=50.
If n=1000000, n^2/2= 500000000000.
n/2=500000.

We can ignore the n/2 term asymptotically (for very large n). Hence the complexity is n^2/2

Insertion sort - complexity

Why do we write O(n^2) instead of n^2?

If you have noticed, when we are counting the total number of insertions, we assume each insertion takes 1 unit of time.

Actually it takes more than that, depending on the programming language and the computer where the program is running.

The cost of an operation is the sum of the costs of memory accesses and computations.

Insertion sort - complexity

- In Python, the numbers are stored in a referential array. To access a number, the program has to first access the reference (1 unit), then the number (1 unit), bring the number to CPU (1 unit), a total of three units of work.

Three units of work has to be done for bringing the number to the left of the current number, and comparison of the two numbers takes 1 unit of work. Then we may have to shift the number to the left (2 units).

Etc. I hope you got the picture.

Insertion sort - complexity

- The costs add up to a number for a particular programming language and computer. That number is different for a different programming language and computer.

We don’t know the language and computer when we are designing an algorithm. So by O(n^2) we mean n^2 multiplied by a constant.

We will discuss this big-oh notation in more (mathematical) details soon.

Insertion sort - complexity

We just discussed *time complexity*. There is another kind of complexity called *space complexity.* How much memory space does an algorithm require?

Every algorithm requires *O(n)* space when n is the size of the input. We must store all the inputs in memory.

Note that we are again using big-oh, since we don't know the computer or the language. We don't know how many bytes will be required for storing a number.

Insertion sort - complexity

We want to know how much extra storage is required.

In case of Insertion Sort, we only need a few extra variables, for example, for the loop index, for copying the current element etc. So the space requirement is O(n), we can ignore the extra variables.

There is a curious phenomenon in algorithms that the time complexity can be reduced by using more space, or vice versa, known as the *space-time tradeoff.*